How Much History Does a Recommender Need?

Training-window length, quality and cost on the Netflix Prize ratings

Navya S G

November 2026

Abstract

Collaborative filtering models are usually trained on all the history a service has kept, although tastes and catalogs change and training cost grows with every month retained. We ask how much history three recommenders actually need. Popularity, Spark preference ALS and a two-tower neural model are trained on windows of 1 to 24 months and on all history before three monthly cutoffs in 2005, and are scored on the following month of the Netflix Prize ratings with NDCG@10 against the full catalog. Every plan was registered before it ran. Popularity and ALS do best on the most recent month: ALS on 1 month scores 1.68 times its all-history score and trains in about an eighth of the time. The two-tower model needs about 6 months, which reach 98.0% of its 12-month score, and no model gains from history beyond 12 months. The two-tower model scores above both baselines at every window and test month. Two supporting studies measure the systems around the models: Spark needed every thread of a 4-thread machine to beat a single-threaded program, and the served model met a 100 ms latency target up to 400 simultaneous users on one laptop.

1 Introduction

A recommender that keeps every rating ever made pays for it twice. Training time grows with the number of rows, and old ratings may describe tastes that users no longer hold or films that are no longer watched (Koren, 2009). Yet the length of the training history is rarely treated as a variable: models are trained on what is available, and evaluated on a random sample of the same period.

This report treats history length as the variable under study. It asks how much interaction history a collaborative filtering model needs, what each additional month costs in compute, and whether a neural model needs more or less history than matrix factorization. The setting is the Netflix Prize data (Bennett and Lanning, 2007), which has about 100 million dated ratings, so a window can be cut at any month and its cost measured.

The contributions are:

- a temporal comparison of three models over six training windows and three test months, with every window scored on the same users and the same removed films;
- a registered decision rule for "history needed" and its answer for each model;
- the cost of each window, and a check of whether storing data by month makes short windows cheaper to train;
- two supporting measurements, of Spark's scaling against a single-threaded baseline and of the served model under load, each with its hardware named.

2 Related work

Time in collaborative filtering. Koren (2009) showed on this dataset that user and item biases drift, and modeled the drift explicitly. This work asks a simpler question that such models leave open: if a model is not told about time, how much history should it be given?

Evaluation. Ji et al. (2023) show that random and leave-one-out splits let models train on interactions from after the test point, and that model rankings change with the amount leaked. Krichene and Rendle (2020) show that metrics computed against a sample of negative items can reverse model orderings, so this work ranks the full catalog. Ferrari Dacrema et al. (2019) found that many neural recommenders did not beat well-tuned simple baselines, and Rendle et al. (2020) that a dot product is hard to beat. Both motivate the strong baselines here, including popularity over the most recent month.

Models. ALS for explicit ratings (Zhou et al., 2008) and for implicit preference (Hu et al., 2008) are standard matrix factorization baselines. Two-tower models encode users and items separately and score them with a dot product (Covington et al., 2016); the one here is trained with an in-batch softmax and the popularity correction of Yi et al. (2019).

Systems cost. McSherry et al. (2015) define COST, the configuration that outperforms a single thread, and found that many distributed systems need many cores to beat one competent thread. We measure Spark's COST for this ALS job.

Privacy. Narayanan and Shmatikov (2008) re-identified subscribers in this dataset from a few known ratings, which shapes how the data is handled here (Section 6).

3 Data and method

Data. The four training files hold 100,480,507 ratings by 480,189 users of 17,770 films, dated from 1999-11-11 to 2005-12-31, with no duplicate user and film pairs. They are stored as Parquet in one folder per rating month. Ratings are skewed towards the end: 53.0% are from 2005, so a window's length says little about its size, and every result reports rows as well as months. November and December 2005 are excluded from testing, because ratings held back for the competition and an 88.6% fall in new users between October and December distort them.

Split. The test months are August, September and October 2005. For a test month starting at cutoff c, a window of m months trains on ratings from c minus m months up to c, and the model is scored on ratings in the month after c. Windows are 1, 3, 6, 12 and 24 months, and all history since November 1999. Settings were tuned once on a July 2005 cutoff, on the 12-month window, and then frozen for every window. Training data is strictly earlier than test data, and tests in the code assert it. A random split would have hidden the effect under study: on one cutoff and seed, preference ALS scored 3.45 times higher on a random split than on the temporal one.

Comparable windows. A shorter window contains fewer users, and could recommend films a user rated before the window. To keep scores comparable across windows, every window at a cutoff is scored on the same users, those who rated anything before the cutoff, and each user's earlier films are removed from their list however short the window. Users absent from a window receive

that window's most popular films, as a deployed service would give them. New users are reported separately and excluded here.

Metric. For each user, the model ranks all films not rated before the cutoff, and a test rating of 4 or 5 stars counts as relevant. The primary metric is NDCG@10, normalized by the best score possible at 10, so a perfect list scores 1. Up to 50,000 eligible users are sampled per arm with a fixed seed. Recall@10 was also recorded. It picks the same best window as NDCG for popularity and ALS at every test month, while for the two-tower model its best window varies among 12 months, 24 months and all history.

Models. *Popularity* counts each film's ratings in the window. *Spark preference ALS* (Hu et al., 2008) learns which films a user chose to rate, with rank 20, regularization 0.01, confidence 3 and 10 iterations. In an earlier registered comparison, preference ALS beat ALS on ratings (Zhou et al., 2008) at every temporal cutoff, so it is the ALS baseline. *The two-tower model* has no per-user ID: the user tower averages embeddings of up to the 50 most recent films the user rated inside the window, on days strictly before the example, and the film tower adds a film ID embedding to embeddings of its release year and title words. It uses 64 dimensions, a batch of 4,096, 3 epochs, learning rate 0.001 and temperature 0.1. Because its input comes only from the window, a 1-month model cannot see older history. ALS and the two-tower model use two seeds; popularity has none.

Registration and decision rules. The windows, cutoffs, seeds, metric and predictions were frozen in a plan before the first run, and every published number is tied to its run and commit in a claims ledger. *History needed* is the shortest window scoring at least 95% of the model's best window at every test month, using the mean over seeds. Two models *differ clearly* at a window only if one is higher at every test month. This paired rule replaced an earlier rule based on overlapping ranges, because ranges across months mostly measure how hard each month is.

Hardware. Popularity and ALS ran on a free Kaggle CPU notebook with 4 cores and 31.3 GB, with Spark in local mode. The two-tower model ran on a Kaggle GPU notebook with one Tesla T4. Times are compared only within a model.

4 Results

4.1 History needed

Table 1 and Figure 1 give the main result. The answer depends on the model.

For popularity and ALS, more history is worse. Both do best on the most recent month at every test month, and for ALS every longer window scores lower at every test month. ALS on 1 month scores 1.68 times its all-history score. At 1 month, ALS and popularity do not differ clearly: popularity is higher in August and ALS in October. From 3 months up, ALS is higher at every test month.

The two-tower model needs about 6 months. It improves up to 6 months and is then flat: its 6-month score is 98.0% of its 12-month score, and the 1- and 3-month windows fall below the 95% line in at least one month. It does not gain beyond 12 months.

The neural model needs more history than ALS, and scores higher at every window. By the paired rule, the two-tower model is above both baselines at every window and test month. Its

Table 1: Mean NDCG@10 over three test months (and two seeds for the trained models). Rows are the mean training rows across test months. Bold marks the history each model needs under the registered rule.

Months	Rows (M)	Popularity	Spark ALS	Two-tower
1	5.0	**0.0837**	**0.0793**	0.0913
3	15.1	0.0609	0.0736	0.0968
6	28.2	0.0500	0.0620	**0.1007**
12	49.2	0.0334	0.0527	0.1028
24	71.8	0.0308	0.0489	0.1026
all	84.3	0.0305	0.0471	0.1010

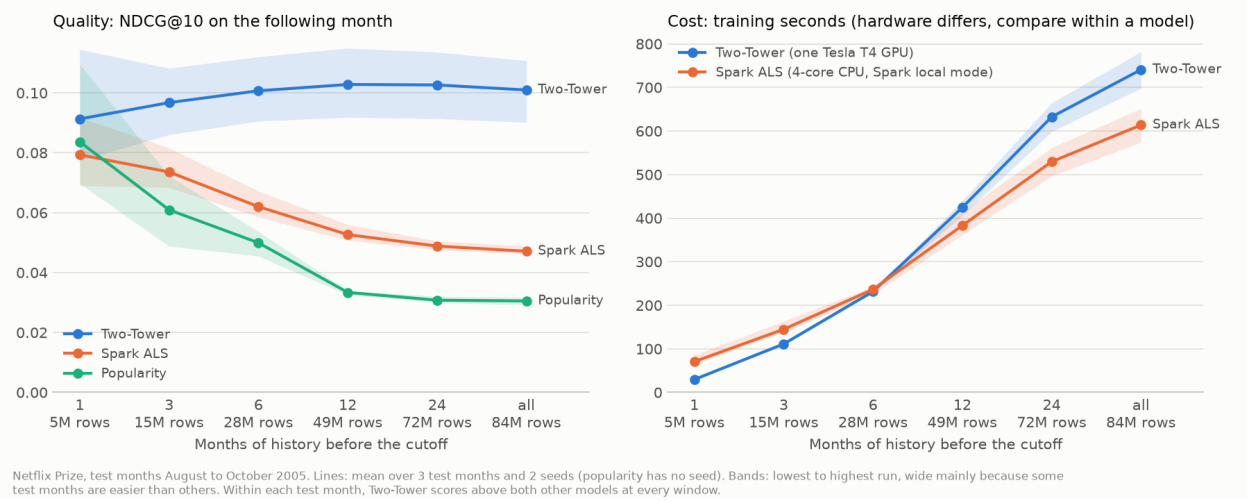


Figure 1: Quality (left) and training time (right) against months of history before the cutoff. Bands span the lowest to highest run; they are wide because some test months are easier than others. Within each test month, the two-tower model scores above both other models at every window. Training times come from different hardware and are compared only within a model.

smallest margin is 0.0038, over popularity on 1 month in August 2005, which is more than four times either arm's standard error across users.

All five registered predictions held. The result is less surprising in light of an earlier correction: before this experiment, ALS had been compared only with popularity over 12 months, which it beat. A diagnostic run then showed that popularity over the last month alone beat both tuned 12-month ALS models, at 0.0806 against 0.0491 for Spark ALS, 1.64 times as high. Beating long-window popularity had been a weak test, and every later comparison included recent popularity.

4.2 What each month costs

Training time grows with rows, not months. Spark ALS on all history took 8.68 times as long as on 1 month (613.7 s against 70.7 s) for a lower score. For the two-tower model, 12 months took 1.83 times as long as 6 (424.4 s against 231.6 s) for a score within 5%.

Storing ratings by month makes short windows cheaper to *read*: in a registered benchmark, monthly partitions cut the read time of a 3-month window by 75.6% to 76.7% against flat Parquet.

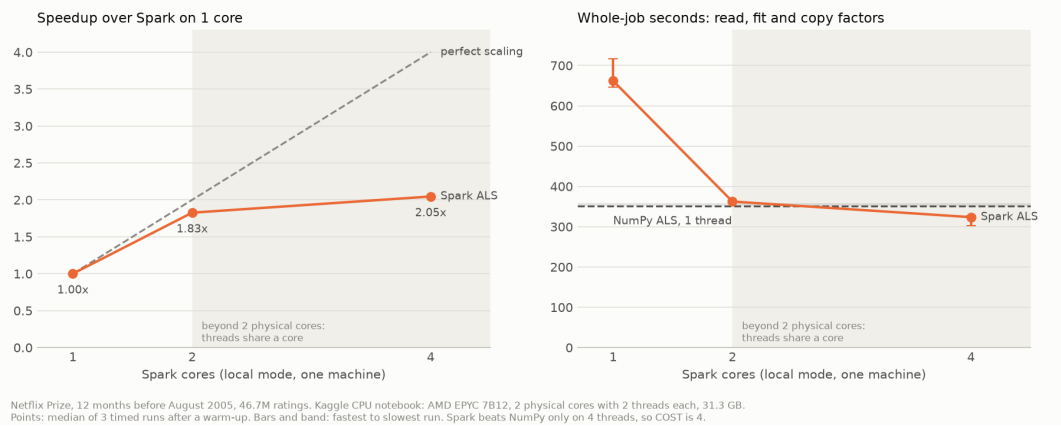


Figure 2: Whole-job time for ALS on the 12 months before August 2005 (46,691,107 ratings), on a Kaggle CPU notebook with 2 physical cores of 2 threads each. Spark runs in local mode.

They did not clearly shorten *training*: whole-job savings had overlapping ranges, with medians of 1.2% on 3 months and 3.5% on 12 months, because reading is a small part of an ALS job. The saving from a short window comes from training on fewer rows: a 3-month job took 65.1% less time than a 12-month job. We therefore do not claim that partitioning shortens training.

4.3 Does Spark pay for itself on one machine?

Because the main experiment used Spark, we measured whether it is needed. On the 12 months before August 2005, with 1, 2 and 4 threads in local mode and three timed runs each (Figure 2), Spark ALS took a median of 662.0 s on 1 core, 362.5 s on 2 and 323.6 s on 4 threads: a speedup of 1.83 on the 2 physical cores and 2.05 on all 4 threads, which share those cores. Single-threaded NumPy ALS doing the same job took 351.4 s. Spark on one core took 1.88 times as long, on 2 cores it was still 3.2% slower, and only on all 4 threads did it win, by a median of 7.9%. Spark's COST for this job is therefore the whole machine. Its value here is code that would run on a cluster, not speed on one machine. This measures threads in one process, not a network of machines.

4.4 Serving the model and its limit under load

The two-tower model on 6 months before October 2005 was retrained for serving and scored 0.0993, against 0.0994 in the main experiment. Evaluation and serving share one ranking function, and for 5,000 sampled users the service returned exactly the lists the evaluation scored. The service uses NumPy over all 17,770 films, without an approximate index, and runs in a three-node Kubernetes cluster created with kind on one laptop.

A load test registered a target before any measured run: a 95th-percentile latency of at most 100 ms with at most 1% errors. The load generator ran on CPUs separate from the cluster's. With one copy of the service, p95 latency was 6.3 ms at 50 users and 72.4 ms at 400 users, and the target was missed at 600 (Figure 3). A second copy did not raise that limit. It reached 1.32 times the peak requests per second of one copy, 710.5 against 540.0, below the registered threshold of 1.5, and also missed the target at 600 users. Two of the three predictions failed: we expected one copy to peak near 300 requests per second, and a second copy to add more. One possible reason, not tested, is that one copy already uses more than one CPU: the web framework serves requests on a pool of threads, and NumPy lets other threads run while it computes. All parts shared one laptop, so these

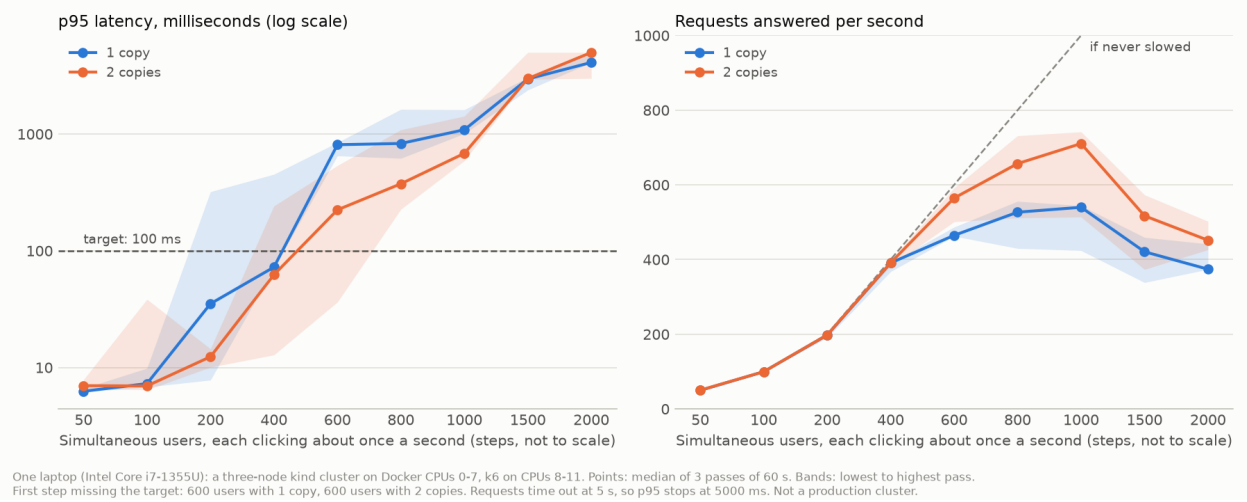


Figure 3: p95 latency (left) and requests answered per second (right) against simultaneous users, for one and two copies of the service on one laptop (Intel Core i7-1355U). Points are medians of three passes of 60 s.

numbers describe that laptop, not a production cluster.

5 Threats to validity

Narrow test period. Three test months, all in late 2005, on one dataset. The finding may not hold for other periods or services, and a second dataset was cut from scope.

Settings. Settings were tuned on the 12-month window, which could favor that window. ALS still did best on 1 month, so this tuning works against the main finding rather than for it. Fixed epochs give short windows fewer training steps, which may understate the two-tower model on short windows.

Exposure. Ratings reflect what Netflix showed its users. Some of the advantage of recent history may come from what the service promoted, which this data cannot separate.

Offline measurement. NDCG on held-out ratings rewards predicting what users rated, not what they would have enjoyed, and favors popular films. No online test was possible.

Seeds and repeatability. Two seeds per trained arm. GPU training was not bit-for-bit repeatable across runs with the same seed, while NumPy ALS was.

Hardware. Times come from shared, free notebooks and one laptop. Models on different hardware are not compared by speed, Spark ran in local mode, and the load test's virtual users wait for each answer, which understates queueing compared with real arrivals.

6 Ethics and data handling

The Netflix Prize data was released with subscribers' identities removed, and Narayanan and Shmatikov (2008) showed that a few known ratings and dates could identify a subscriber's record. This project therefore does not redistribute the data or any sample of it. The repository holds code, registered plans and aggregate results only. The model bundle served in the cluster holds user IDs and their rated films, so it is kept out of the repository and the container image, the service never logs a user ID, and it returns film IDs without titles. Tests and continuous integration use synthetic data. Recommenders trained on ratings also tend to favor popular films; this work measures accuracy only and makes no fairness claim.

7 Conclusion and future work

How much history a recommender needs depends on the model. Popularity and ALS did best on the most recent month and grew worse with more, while training time grew with every row. The two-tower model needed about 6 months and gained nothing beyond 12. For all three models, history beyond a year cost compute and bought no accuracy on these test months.

Natural next steps are a breakdown by user tenure and film popularity, more test months and a second dataset, tuning each window separately, training for a fixed number of steps rather than epochs, and testing whether an approximate index or several machines change the serving and scaling results.

Reproducibility. The code, registered plans, results, claims ledger and lab notebook are public. `trs reproduce --sample` runs the whole method on synthetic data with a planted change in user tastes on any laptop and checks that it is found. The full-data commands are listed in `docs/REPRODUCING.md`, each with the commit its results came from.

References

James Bennett and Stan Lanning. The Netflix Prize. In *Proceedings of KDD Cup and Workshop 2007*, 2007.

Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, 2016. doi: 10.1145/2959100.2959190.

Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019. doi: 10.1145/3298689.3347058.

Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining*, pages 263–272, 2008. doi: 10.1109/ICDM.2008.22.

Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. A critical study on data leakage in recommender system offline evaluation. *ACM Transactions on Information Systems*, 41(3):75:1–75:27, 2023.

Yehuda Koren. Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 447–456, 2009. doi: 10.1145/1557019.1557072.

Walid Krichene and Steffen Rendle. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020. doi: 10.1145/3394486.3403226.

Frank McSherry, Michael Isard, and Derek G. Murray. Scalability! But at what COST? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*. USENIX Association, 2015.

Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy*, pages 111–125, 2008. doi: 10.1109/SP.2008.33.

Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. Neural collaborative filtering vs. matrix factorization revisited. In *Proceedings of the 14th ACM Conference on Recommender Systems*, 2020. doi: 10.1145/3383313.3412488.

Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019. doi: 10.1145/3298689.3346996.

Yunhong Zhou, Dennis Wilkinson, Robert Schreiber, and Rong Pan. Large-scale parallel collaborative filtering for the Netflix Prize. In *Algorithmic Aspects in Information and Management (AAIM)*, volume 5034 of *Lecture Notes in Computer Science*, pages 337–348. Springer, 2008. doi: 10.1007/978-3-540-68880-8_32.